Arduino for Embedded Systems - Beginner Level

Lesson 8: Introduction to IoT with Arduino

Introduction to IoT with Arduino - Study Notes

Key Concepts

1. **ESP8266 Wi Fi Module** – a low cost microcontroller with built in TCP/IP stack for connecting Arduino style boards to the internet.

2. **RESTful Web Endpoint** – a URL that accepts HTTP requests (usually `POST`) to receive data, often returning JSON.

3. **Sensor Integration** – reading analog or digital sensors (e.g., temperature, humidity, light) and formatting the data for transmission.

4. **HTTP POST Request** – sending data from the ESP8266 to a server using the `POST` method, including headers and a payload (typically JSON).

5. **Basic Security Practices** – using HTTPS (or at least API keys) and avoiding hard coded credentials in source code.

Summary

The Internet of Things (IoT) lets physical devices exchange data over the internet. In this lesson you learned how to use an ESP8266 module—commonly paired with Arduino boards—to read sensor values and push them to a remote web service. The workflow consists of three steps: (1) wiring the sensor to the ESP8266 (or an Arduino that communicates with the ESP8266 via Serial), (2) establishing a Wi Fi connection, and (3) formatting the sensor reading as JSON and sending it with an HTTP `POST` request to a predefined endpoint.

Understanding the HTTP request structure and the ESP8266's networking library (`ESP8266WiFi` and `ESP8266HTTPClient`) is essential. You also saw how to handle common pitfalls such as connection time outs, JSON formatting errors, and the importance of keeping Wi Fi credentials and API keys out of public repositories. With these basics, you can expand the project to multiple sensors, use MQTT instead of HTTP, or add OTA (over the air) updates.

Important Points

- **Hardware Setup**

- ESP8266 (e.g., NodeMCU, Wemos D1 Mini) can be programmed directly with the Arduino IDE.

- Connect sensor power (VCC/GND) and data pin (e.g., `A0` for analog temperature sensor) to the ESP8266.

- **Wi Fi Connection**

```cpp
WiFi.begin(ssid, password);
while (WiFi.status() != WL_CONNECTED) {
```

```
    delay(500);
}
```

    • **Reading a Sensor**

```cpp
int raw = analogRead(A0);
float voltage = raw * (3.3 / 1023.0);
float temperatureC = (voltage - 0.5) * 100; // example for LM35
```

    • **Creating JSON Payload**

```cpp
String payload = "{\"temp\":";
payload += String(temperatureC, 2);
payload += "}";
```

    • **Sending HTTP POST**

```cpp
HTTPClient http;
http.begin("http://example.com/api/sensor"); // use https:// for TLS
http.addHeader("Content-Type", "application/json");
int httpResponseCode = http.POST(payload);
http.end();
```

    • **Error Handling**

- Check `httpResponseCode` (>0 = success, <0 = client error).
- Re connect Wi Fi if `WiFi.status()` changes.

    • **Security Tips**

- Store SSID/password in a separate header file (`secrets.h`) and add it to `.gitignore`.
- Use HTTPS or a token based API key in the request header.

    • **Power Considerations**

- ESP8266 draws ~70 /mA when transmitting; ensure a stable 3.3 /V supply.
- Use deep sleep (`ESP.deepSleep()`) for battery powered deployments.

Examples

1. Simple Temperature Logger

Goal: Read an LM35 temperature sensor and POST the temperature to `http://myserver.com/api/temp`.

```cpp
#include <ESP8266WiFi.h>
#include <ESP8266HTTPClient.h>
const char* ssid     = "YOUR_SSID";
```

```
const char* password = "YOUR_PASS";
void setup() {
  Serial.begin(115200);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWiFi connected");
}
void loop() {
  // 1. Read sensor
  int raw = analogRead(A0);
  float voltage = raw * (3.3 / 1023.0);
  float tempC = (voltage - 0.5) * 100.0;   // LM35 conversion
  // 2. Build JSON
  String json = "{\"temperature\":";
  json += String(tempC, 2);
  json += "}";
  // 3. POST to server
  if (WiFi.status() == WL_CONNECTED) {
    HTTPClient http;
    http.begin("http://myserver.com/api/temp");
    http.addHeader("Content-Type", "application/json");
    int code = http.POST(json);
    Serial.printf("POST code: %d\n", code);
    http.end();
  }
  delay(60000); // send once per minute
}
```

Explanation: The sketch connects to Wi Fi, reads the analog voltage from the LM35, converts it to Celsius, wraps the value in a JSON object, and posts it. The loop runs every minute to avoid flooding the server.

2. Multiple Sensors with JSON Array

Goal: Send temperature and light intensity (via an LDR) together.

```cpp
// ... (WiFi setup same as before)
void loop() {
  float tempC = readTemp();          // function from previous example
  int   light = analogRead(A0);      // assume LDR on A0
  String json = "[";
```

```
  json += "{\"sensor\":\"temp\",\"value\":" + String(tempC,2) + "},";
  json += "{\"sensor\":\"light\",\"value\":" + String(light) + "}";
  json += "]";
  if (WiFi.status() == WL_CONNECTED) {
    HTTPClient http;
    http.begin("https://api.myserver.com/data");
    http.addHeader("Content-Type", "application/json");
    http.addHeader("X-API-Key", "YOUR_API_KEY"); // optional auth
    int code = http.POST(json);
    Serial.println("Sent: " + json);
    Serial.printf("Response: %d\n", code);
    http.end();
  }
  delay(30000); // every 30 seconds
}
```

Explanation: This example demonstrates building a JSON **array** containing two sensor objects, adding an API key header for authentication, and using HTTPS (requires proper certificate handling on the ESP8266).

Quick Review

1. **What library is needed to make HTTP requests from an ESP8266?**
2. **How do you convert an analog reading (0 1023) to a voltage on a 3.3 /V ESP8266?**
3. **Why is it recommended to keep Wi Fi credentials out of the main sketch file?**
4. **What HTTP method is typically used to send sensor data to a server, and why?**
5. **Name one power saving technique for battery operated ESP8266 projects.**

Further Reading

- **MQTT Protocol** – lightweight publish/subscribe messaging for IoT.
- **TLS/SSL on ESP8266** – using `BearSSL` or `WiFiClientSecure` for encrypted communication.
- **OTA Updates** – updating firmware over Wi Fi with the Arduino OTA library.
- **Node RED & InfluxDB** – building a local dashboard to visualize incoming sensor data.
- **Deep Sleep & RTC** – extending battery life by putting the ESP8266 into low power mode between transmissions.